

Reflection

Last updated: June 24, 2026

Summary

Reflection AI is looking to source a set of SWE tasks created from open-source repos. These tasks should be SWE-Bench style tasks including problem statements, Dockerfiles, fail2pass tests, and gold patches. We are looking to source high-quality and end-to-end validated tasks across a variety of languages and repo types.

As described in this spec, all parts of the problem (including problem statement, dockerfile, fail to pass tests, and the test patch + golden patch) must be expert-written and verified for quality).

Timeline

Jun 25 – Please submit a set of repos which you would like to get tasks from. Reflection must sign off + select from the list of repos.

Jul 1 – 2,500 tasks delivered

Jul 8 – 5,000 tasks delivered

Jul 15 – 7,500 tasks delivered

Jul 22 – 7,500 tasks delivered

Jul 29 – 7,500 tasks delivered

Difficulty

We are looking to target the following distribution:

Medium: 50% (solvable by a frontier model such as Claude Opus)

Difficult: 50% (not solvable by frontier or open-source models)

To put into Pass@N terms:

Medium Tasks: GPT5.5 or Claude Opus 4.8 solves $\leq 4/8$ attempts

Hard tasks GPT 5.5 or Claude Opus 4.8 solves $\leq 2/8$ attempts.

To measure difficulty, please use the model in their respective harness (Codex + Claude Code).

Please run `pass @ 8` and include the results in the metadata. Given the high cost of determining the difficulty of the tasks, we suggest that a cheaper model is used to perform an initial difficulty assessment (e.g., tasks that are easily solvable by Sonnet or Haiku are unlikely to be hard for Opus, so these can be filtered out).

Please instruct annotators to pick difficult tasks based on areas/repos that they are an expert in and tasks should be naturally difficult, for example:

Good Difficult Tasks: Hard cybersecurity or research problems, large feature additions, complex debugging, root-cause analysis.

Bad Difficult Tasks: combining multiple unrelated tasks into a single task, tasks that are difficult only because requirements are vague or underspecified.

Submission Requirements

Upload tasks in Harbor format. The image URI should be in task.toml. **Please upload to the GCS folder (gs://turing-reflection-swe-alfa/) provided by Reflection in the following format:** [delivery-date-mm-dd-yyyy]-[task-count]/[task-id]/[harbor format task]. For example:

None

```
turing-reflection-swe-alfa/
├── 06222026-5000/
│   ├── django__django-11099/
│   │   ├── task.toml (including image URI as docker_image= on GCS, see example below)
│   │   ├── instruction.md
│   │   ├── enviroment/
│   │   │   ├── Dockerfile
│   │   │   ├── problem_statement.md
│   │   ├── solution/
│   │   │   ├── solve.sh
│   │   │   └── golden.patch
│   │   └── tests/ (tests folder can only contain up to 3 files: grade.py, config.json, and test.sh)
│   │       └── test.sh
```

Resubmission.

For resubmission, please resubmit failing tasks in a **new** directory with the number of tasks and -fix suffix. For example, in the above 5000 tasks, if 750 tasks are failing and resubmitted they should be resubmitted as new tasks with the same task ids under 06222026-750-fix

None

```
turing-reflection-swe-alfa/
├── 06222026-5000/ ...
├── 06222026-750-fix/
│   ├── django__django-11099/
```

```

    └─ task.toml (including image URI as docker_image= on GCS, see example
below)
    └─ instruction.md
    └─ enviroment/
        └─ Dockerfile
        └─ problem_statement.md
    └─ solution/
        └─ solve.sh
        └─ golden.patch
    └─ tests/ (tests folder can only contain up to 3 files: grade.py,
config.json, and test.sh)
        └─ test.sh

```

Please ensure that images are on Google AR as pass the following criteria:

- No future commits
- No reflog
- No remote
- No large Git directory (less than 100 mb)
- .git must be present
- HEAD must equal base commit

Please ensure that tasks fit the following criteria:

- Harbor format, including a separate gold patch in the solution folder named golden.patch
 - There should be both a [solve.sh](#) (should be standard across all tasks, and be a simple patch application) and gold patch available.
- All tasks require no internet access. This includes setup, dependencies, which should all be baked into the image.

Tasks must be uploaded to a GS folder and Google AR directory as defined before the pipeline begins. Each top-level should be formatted as follows:

[delivery-date-mm-dd-yyyy]-[task-count]/[task-id]/[harbor format task]

All tasks must be runnable on Reflection infra. We will apply the gold patch file and run [test.sh](#), tests should pass via this pipeline. Note that [solve.sh](#) is not used in our workflow

Please include in task.toml information such as the following:

- **Identity:** vendor, name, version, contact.

- **Artifacts:** immutable, pinned references (e.g. image digest).
- **Resource budget:** memory, CPU, per-episode timeout, scratch-disk.
- **Network:** required endpoints, and whether episodes run with no egress. **Tasks should be offline only for eval, setup, and agent runtime.**
- **Tasks:** how to enumerate and select tasks, and what each task carries – instruction, environment, verification/oracle, and golden solution (cf. the Harbor task format).
- **Golden solutions/trajectories:** to validate verifier

Here is an example task.toml.

JSON

```
schema_version = "1.3" # Harbor format version (NOT task version)

# — [task] — Identity (per-task authorship) —————
[task]
name          = "vendor-ai/swe-bench-problems" # org/name; registry-unique
description    = "SWE-Bench Style Training Problems"
authors       = [{ name = "Vendor AI", email = "jeffrey@vendorai.com" }]
keywords      = ["code", "swe"]
metadata      = { "vendor" = "Vendor AI", "deal_id" = "XXXX" }

# — [environment] — Artifacts + Resources + Network baseline —————
[environment]
docker_image   = "registry.acme.example/gpqa-suite@sha256:abc123def..." #
# pinned image URI (prebuilt mode)
os             = "linux"
cpus           = 2                # resource: CPU
memory_mb      = 2048             # resource: memory
storage_mb     = 5120             # resource: ephemeral scratch disk
gpus           = 0
gpu_types      = []
build_timeout_sec = 600           # resource: build ceiling
workdir        = "/workspace"
network_mode   = "no-network" # network baseline: public | no-network |
allowlist
allowed_hosts  = []              # (for allowlist; empty = deny all egress)
# env          = { TASK_SEED = "42" }
# instance seed for stochastic tasks (convention)
# healthcheck  = { command = "...", interval_sec = 30, timeout_sec = 5, retries =
3 }
```

```
[environment.tpu] # GKE only, optional
# type         = "v6e"
# topology     = "2x4"
```

```

# — [agent] — per-episode resource ceiling + phase network —————
[agent]
timeout_sec = 300 # resource: per-episode timeout
# network_mode = "allowlist" # phase override (requires
dynamic_network_policy)
# allowed_hosts = ["api.openai.com"]
# user = "agent"

# — [verifier] — oracle + isolation —————
[verifier]
timeout_sec = 120
# environment_mode = "separate" # isolated grading sandbox
# environment = { docker_image = "...", memory_mb = 1024, network_mode =
"no-network" }
# collect = [ # tamper-resistant evidence sidecars
# { command = "pytest -v", service = "app", timeout_sec = 60 },
# ]

# — [solution] — golden solution (script) —————
[solution]
# actual golden solution: solution/solve.sh
# actual golden solution patch: solution/golden.patch
# env = { ... }

# — [metadata] — free-form —————
[metadata]
difficulty_explanation = "expert-level cybersecurity problem"
pass_at_k_gpt_5_5 = 3/8
pass_at_k_opus_4_8 = 3/8
category = "security"

# Multi-step tasks add [[steps]], each with its own agent/verifier/min_reward.
# Root-level: source, multi_step_reward_strategy, artifacts (OUTPUT snapshots –
not inputs).

```

Languages

We are looking to cover an distribution of the following languages: C, C++, Go, Java, JavaScript / TypeScript, PHP, Ruby, Rust.

JavaScript / TypeScript: 30%

Python: 22%

C / C++: 20%

Java: 11%

C#: 8%

PHP: 5%

Go: 4%

Ruby: 3%

Rust: 2%

Task Quality

In addition to being generally high-quality tasks, tasks should follow the following quality requirements:

For tests, please ask annotators to write a comprehensive suite of fail to pass tests to prevent reward hacking (e.g. 10+).

Tasks can be based on a PR, commit, or issue, be derivations from an existing PR, or be net-new. For each, please mark tasks from their source. Please provide examples in the first quick-hits of each task type that will be created in the pipeline. Less than 50% of tasks should be net-new.

Please consider the following feedback for contributors in future deliveries.

- Try to avoid giving explicit instructions e.g. "Where to look: [list of relevant files]". Being clear about test assumptions and edge cases is reasonable
- Please do not leak any hidden test information e.g. nothing about the tests
- Problem statements should discuss the problem itself rather than how to implement the solution

As a general frame of reference, we are looking for feature implementation and bug fixes which have an average LoC ~350 and edit multiple files.

Tasks should also follow the following quality guidelines:

Human-authored: Task descriptions, test logic, metadata, and explanations must be written and reviewed by human experts. LLM-generated content is not acceptable unless explicitly disclosed and substantially rewritten by an expert.

Difficult-tier justified: The task must genuinely warrant a difficult classification. Difficulty must arise from reasoning complexity, cross-module understanding, subtle behavioral differences, or deep domain knowledge — not from artificially constrained environments, misleading task descriptions, or excessive boilerplate.

Realistic: The task must represent a plausible software engineering workflow — debugging a regression, implementing a feature, fixing an edge-case bug, resolving a performance issue, or updating behavior to match a specification. It must not be a contrived puzzle with no realistic analogue.

Agentic: The task must require meaningful code exploration, file manipulation, test execution, debugging, or multi-step reasoning. It must not be solvable by a single edit or a zero-shot code generation without understanding the existing codebase.

Solvable: A correct solution must be achievable within the configured time and resource limits by an expert software engineer familiar with the relevant language and domain.

Verifiable: Completion must be judged by deterministic, programmatic tests that verify the final behavior of the modified codebase. Subjective grading is not permitted. For this dataset, we want a significant amount of F2P tests: e.g. the average task should have ~10-20 F2P tests that sufficiently test function behavior and prevent reward hacking.

Outcome-based: Tests must verify observable behavior or final state, not the specific file edited, function renamed, or commit message written.

Deterministic: Re-running the task setup, reference solution, no-op, and verifier under the same configuration must produce consistent outcomes.

Secure: Task files must not contain malicious code, credential exfiltration, prompt injection, host escape attempts, or cross-task interference.

Anti-cheat robust: The task must not expose ground-truth patches, expected test outputs, or solution logic to the agent. The agent must not be able to complete the task by shortcutting the intended engineering work.

Clear objective: The task goal must be directly and unambiguously stated in terms of expected behavior, not in terms of what code to change.

Concise: The task problem statement must include only the information needed to understand and reproduce the issue. It must avoid unnecessary filler, roleplay framing, or generic tool descriptions.

No solution hints: The task problem statement must not reveal the root cause, the intended fix, the file to edit, or the algorithm to apply, unless that information was present in the original issue.

No unsupported claims: The description must not claim that behaviors, APIs, or features exist unless they are actually present in the pinned repository state.

Structured output specified when applicable: If the task requires changes to a specific API response format, schema, or output structure, the exact expected format must be documented in the issue or in a clearly referenced file in the environment.

Environment description accurate: Any description of the repository's structure, relevant modules, entry points, or service configuration must match the actual state of the pinned repository.

Present and executable: The solution directory must contain either an executable `solve.sh` or a valid unified diff `patch.diff` that applies cleanly to the base commit.

Correct: Applying the reference solution to a clean environment must resolve the issue and produce a passing verifier result.

Matches the canonical fix: The reference solution must correspond to the actual fix committed to the upstream repository, not an alternative fix invented for the benchmark, unless the upstream fix is unavailable or inappropriate for benchmark use and a justification is documented.

Clear: The solution must be readable and organized enough for reviewers to understand the intended approach and the root cause being addressed.

Non-trivial: The solution must demonstrate that the task requires meaningful engineering judgment. It must not reduce to inserting a single character, commenting out a line, or making a change that requires no understanding of the codebase.

Not hardcoded: The solution must fix the underlying issue through legitimate code changes. It must not simply make the failing tests pass by special-casing them or hardcoding expected values.

No privileged knowledge: The solution must not rely on information unavailable to a real agent, such as knowledge of the verifier's exact assertions or expected output values not present in the issue or codebase.

Consistent with issue: Every change made by the solution must be relevant to and permitted by the task described in the issue.

Consistent with tests: The solution must satisfy all verifier checks without calling, modifying, or depending on the verifier.

No unnecessary changes: The solution must not make unrelated edits, broad refactors, style cleanups, or changes not required to resolve the issue.

Deterministic: Applying the solution in a clean task environment must consistently produce the same passing final state.

Deterministic: Tests must use deterministic inputs, seeds, and assertions. Nondeterministic test ordering must be fixed or controlled.

Fast enough: Tests must complete within the configured timeout. Long-running test suites must be scoped to the relevant subset, with a justification for any tests included beyond those directly relevant to the issue.

Functional verification: Tests must verify actual behavior by executing code, invoking the corrected API, querying services, or inspecting runtime outputs. Tests must not rely on patch structure, diff format, line numbers, or source code keyword matching.

Regression included: The verifier must include a test that directly reproduces the failure described in the issue — a test that passes after applying the reference solution and fails on the pre-patch environment.

No extra requirements: Tests must not assert requirements absent from the issue or from clearly referenced specification documents.

No missing requirements: Every material behavioral requirement in the issue must have corresponding test coverage.

Correct expected results: Each assertion must map inputs to expected outputs correctly. Expected values must be derivable from the issue, the repository documentation, or the upstream fix.

Fixtures committed: Any test fixtures, mock responses, or input data files must be committed or baked into the verifier image. Generated fixtures must be deterministic and reproducible.

Independent from reference solution: Tests must not call the reference solution, import solution code, compare against files created only by the solution, or depend on solution-side artifacts.

Agent-tamper resistant: The agent must not be able to modify verifier code, expected outputs, ground truth, or scoring logic. Ground-truth files used to compute scores must be re-copied into the environment to prevent agent modification.

Domain knowledge required: The task must require knowledge of the relevant framework, protocol, language feature, or library internals that goes beyond surface-level API familiarity. A generalist who has never used the relevant stack must not be able to solve it by reading the issue description alone.

No trivial analogues: The task must not be trivially solvable by copying a nearly identical fix from elsewhere in the same codebase. If such analogues exist, the task must require meaningful adaptation, not mechanical transcription.

Benchmarked difficulty validated: The task's difficult classification must be validated by benchmark results showing the required pass rate thresholds across the required agent set.

The **Dockerfile** must use a minimal amount of base images across all tasks. There is flexibility here but the key is that a low number of base images is shared across many tasks.

Task Diversity

The vendor must ensure that the pairwise cosine similarity between the following pairs is less than **0.90**, as measured using `all-MiniLM-L6-v2`:

- Any pair of `problem statement` files
- Any pair of `solution/golden.patch`

The following constraints apply to the distribution of source repositories:

- No single repository may contribute more than **10%** of the total tasks in the dataset.
- No single repository owner (GitHub organization or user) may contribute more than **20%** of the total tasks.
- At least **40%** of tasks must come from repositories with more than 1,000 GitHub stars at the time of task creation, to ensure tasks are drawn from actively maintained, real-world codebases.
- Tasks must span at least **8 distinct primary programming languages** across the dataset.
- No single programming language may represent more than **30%** of the total tasks.

Each task must be assigned to the category and subcategory that best describes the dominant engineering work required to resolve the issue. If multiple categories apply, choose the one corresponding to the main objective of the fix, not incidental supporting steps.

Distributional requirements:

- No category may make up more than **20%** of the total tasks.
- No category may make up less than **5%** of the total tasks.
- Within each category, no subcategory may make up more than **20%** of the total tasks in that category.
- Where feasible, each subcategory should be covered. Additional subcategories are permitted where they arise naturally.

Software Engineering — Tasks where the fix or implementation touches application logic, library behavior, API contracts, CLI behavior, or service functionality. Use this when the issue is about a behavioral gap or regression in the software itself, not the surrounding build or infrastructure.

- **Feature implementation** — Issues requesting new application logic, library methods, CLI commands, API endpoints, or service capabilities that were absent.
- **Refactoring and code modernization** — Issues requiring structural cleanup, modularization, deprecated API removal, or framework migration without materially changing behavior.
- **Testing and quality engineering** — Issues requiring tests to be written, repaired, extended, or integrated; includes fixtures, linting, and static checks.
- **Compilers, interpreters, and programming languages** — Issues in parsers, interpreters, compilers, transpilers, type checkers, or language runtimes.
- **Porting and migration** — Issues requiring code or tooling to be adapted across language versions, frameworks, operating systems, or dependency ecosystems.
- **Scripting and automation** — Issues in shell scripts, Python scripts, or glue code used to orchestrate local developer workflows.
- **Web, API, and networking software** — Issues in web services, HTTP clients, API integrations, socket programs, request handling, or networked application logic.
- **Version control and repository operations** — Issues involving Git history, hooks, submodules, monorepo tooling, or repository integration.

Debugging and Repair — Tasks where the central challenge is root-causing a failure rather than adding new functionality. Use this when the issue describes something broken and the fix requires investigation before any code change.

- **Runtime bug repair** — Crashes, exceptions, incorrect outputs, memory errors, or wrong runtime behavior.
- **Test failure repair** — Failed tests due to mismatched expectations, broken fixtures, flaky behavior, or regressions introduced upstream.
- **Build failure repair** — Compilation, packaging, dependency, or build-system errors preventing the project from building.
- **Configuration repair** — Broken config files, environment variables, service settings, or runtime parameters causing incorrect behavior.
- **Performance debugging** — Slow code, excessive memory use, timeouts, inefficient queries, or scalability bottlenecks identified via profiling or measurement.
- **Concurrency and synchronization debugging** — Race conditions, deadlocks, nondeterminism, async failures, or event-loop bugs.
- **Pipeline and orchestration debugging** — CI jobs, data pipelines, scheduled jobs, or multi-step automated workflows that produce incorrect or missing output.

Build, Dependency, and Release Management — Tasks where the build, packaging, or dependency resolution process is itself the main objective — not the application logic it produces.

- **Build system configuration** — Make, CMake, Bazel, Gradle, Cargo, npm, setuptools, or pyproject configuration issues.
- **Dependency and lockfile resolution** — Dependency conflicts, version pinning, lockfile correctness, vendoring, or reproducible installation failures.

- **CI/CD pipelines** — GitHub Actions, GitLab CI, Jenkins, build matrices, test workflows, or deployment pipeline configuration.
- **Container builds** — Dockerfile issues, multi-stage build problems, image size, or build reproducibility.
- **Cross-compilation and platform targeting** — Issues building for a platform, architecture, or runtime different from the build host.
- **Package publishing** — Issues packaging and publishing libraries to PyPI, npm, crates.io, or container registries.
- **Release artifacts** — Version bumps, changelogs, signed artifacts, checksums, or distributable release packaging.

Systems, Infrastructure, and Operations — Tasks where most of the work is system or environment configuration rather than application code. Use this when the issue is about how the software runs or is deployed, not what it computes.

- **OS, process, and service management** — Daemons, systemd units, process supervision, signals, startup behavior, or service lifecycle.
- **Users, permissions, and access control** — Unix permissions, ownership, ACLs, sudo configuration, or access restrictions.
- **Shell and environment configuration** — Shell startup files, PATH, environment variables, dotfiles, or local developer environment setup.
- **Networking configuration** — DNS, routing, ports, firewalls, proxies, TLS configuration, or protocol-level setup.
- **Containers and orchestration** — Docker, Compose, Kubernetes, container networking, volumes, scheduling, or image runtime behavior.
- **Storage and filesystem administration** — Mounts, disks, partitions, quotas, filesystems, or system-level file storage.
- **Scheduling and automation infrastructure** — Cron, systemd timers, queue workers, or automated maintenance jobs.
- **Logging, monitoring, and observability** — Logs, metrics, alerts, health checks, tracing, or operational telemetry.

Data Processing and ETL — Tasks where the goal is correctly reshaped, parsed, or validated data. Use this when the issue concerns data transformation or serialization correctness, not query logic or statistical inference.

- **ETL pipelines** — Multi-stage extract-transform-load workflows producing incorrect or missing output.
- **File format parsing and serialization** — Incorrect reading or writing of CSV, JSON, YAML, XML, Parquet, binary formats, or custom file formats.
- **Tabular transformation** — Issues in joins, pivots, group-bys, normalization, filtering, or aggregation of structured data.
- **Text processing** — Tokenization, normalization, extraction, pattern matching, or transformation of natural-language or log text.

- **Data validation** — Schema checks, quality constraints, deduplication, anomaly detection, or consistency validation.
- **Streaming data processing** — Issues in continuous, incremental, or high-volume data stream processing.
- **Media data processing** — Issues transforming or extracting from images, audio, video, or OCR outputs as data conversion tasks.

Data Querying and Databases — Tasks where the main work is query correctness, query performance, or database administration. Use this when the issue concerns how data is retrieved or stored, not how raw files are transformed.

- **SQL querying** — SELECT queries, joins, filters, aggregations, window functions, CTEs, or DML correctness issues.
- **Analytical queries** — OLAP-style summaries, grouped reports, rollups, cohorts, or time-window query issues.
- **Query optimization** — Indexes, query plans, slow queries, rewrites, partitioning, or execution cost issues.
- **Database administration** — Schema migrations, backups, restores, users, permissions, connection setup, or database configuration.
- **NoSQL and document stores** — Key-value stores, document databases, Redis, MongoDB, or non-relational query correctness.
- **Graph and semantic queries** — Graph databases, RDF, SPARQL, or relationship-centric query issues.

Machine Learning and AI — Tasks involving ML model behavior, inference pipelines, evaluation, or ML system integration. Use this when the issue is about model-facing logic rather than training infrastructure.

- **Model inference and prediction** — Incorrect predictions, classification errors, embedding generation failures, or model output issues.
- **Model evaluation and benchmarking** — Metrics, validation splits, benchmark scoring, calibration, or model comparison issues.
- **Feature engineering** — Feature extraction, selection, encoding, or preprocessing issues for ML models.
- **NLP and language models** — Text classification, extraction, summarization, tokenization, retrieval-augmented workflows, or language-model integration issues.
- **ML serving and deployment** — Inference endpoints, batch inference, model packaging, or deployment pipeline issues.
- **Interpretability and model inspection** — Feature attribution, saliency, probing, or model explanation issues.

Model Training and ML Infrastructure — Tasks where the training run, fine-tuning process, or training infrastructure is central to the issue.

- **Training loops** — Forward passes, backward passes, optimizers, loss computation, gradient updates, or custom training code issues.
- **Fine-tuning** — Issues adapting pretrained models, including parameter-efficient fine-tuning or supervised fine-tuning.
- **Data loading and training pipelines** — Dataset, dataloader, batching, shuffling, augmentation, or preprocessing issues affecting training.
- **Checkpointing and resumption** — Issues saving, loading, resuming, converting, or validating model checkpoints.
- **Distributed training** — Multi-GPU, multi-node, or parallelism-related training failures.
- **Evaluation infrastructure** — Eval harnesses, benchmark runners, scoring pipelines, or model-comparison infrastructure issues.

Security — Tasks where the core goal is security correctness, vulnerability remediation, or security investigation.

- **Cryptography** — Incorrect use of encryption, hashing, signatures, key exchange, or cryptographic primitives.
- **Authentication and authorization** — Issues with tokens, sessions, OAuth, JWT, MFA, permissions, or access control.
- **Vulnerability analysis** — Issues involving injection, path traversal, unsafe deserialization, memory corruption, or insecure defaults.
- **Security hardening** — Secure configuration, least-privilege enforcement, secrets removal, patching, or TLS settings.

Scientific Computing and Domain Science — Tasks involving numerical simulation, scientific modeling, or domain-specific scientific workflows.

- **Numerical methods** — Root finding, numerical integration, interpolation, finite differences, or iterative solver issues.
- **Differential equations and simulation** — ODEs, PDEs, discretization, boundary conditions, or numerical simulation issues.
- **Biology and bioinformatics** — Sequence processing, FASTA/FASTQ handling, genomics pipelines, or computational biology workflows.
- **Signal processing** — Filtering, Fourier transforms, spectral analysis, or DSP pipeline issues.
- **Statistical modeling** — Fitting probabilistic or generative models as part of scientific computation.

Mathematics and Formal Reasoning — Tasks involving exact mathematical computation, symbolic manipulation, or formal verification.

- **Symbolic computation** — Algebraic simplification, symbolic solving, formula manipulation, or computer algebra system issues.
- **Number theory and exact arithmetic** — Primes, factorization, modular arithmetic, big integers, or arbitrary precision arithmetic issues.

- **Computational linear algebra** — Matrices, eigenproblems, decompositions, or linear-system solving issues.
 - **Algorithms and optimization theory** — Algorithmic correctness, graph algorithm bugs, or optimization formulation issues.
 - **Formal verification** — Theorem provers, model checkers, SMT solvers, proof scripts, or mechanically checked correctness.
-

Task objective taxonomy — The task objective describes what the agent is ultimately being asked to accomplish. Assign all labels that genuinely apply to the end goal — multiple labels are permitted. Each label must appear in the metadata of each task.

- **Fix** — Diagnose and repair broken behavior, failing tests, runtime errors, build failures, misconfigurations, regressions, or incorrect outputs.
 - **Implement** — Add new functionality, behavior, commands, scripts, tests, models, queries, or artifacts that did not previously exist.
 - **Refactor** — Restructure existing code, configuration, or artifacts to improve maintainability, organization, or compatibility while preserving intended behavior.
 - **Test** — Create, modify, run, or extend tests, benchmarks, fixtures, harnesses, or evaluation suites to assess behavior or correctness.
 - **Optimize** — Improve performance, memory use, query efficiency, artifact size, or resource usage.
 - **Migrate** — Move code, data, configuration, dependencies, or workflows from one version, format, platform, or framework to another.
 - **Configure** — Set up or modify environment settings, services, permissions, packages, containers, CI jobs, databases, or runtime parameters.
 - **Debug** — Investigate the cause of a failure, discrepancy, slowdown, crash, or unexpected result, with or without requiring a code change.
 - **Validate** — Check correctness, consistency, schema conformance, security posture, test results, or expected behavior.
 - **Build or package** — Compile, bundle, containerize, package, or produce deployable or distributable artifacts.
 - **Analyze** — Inspect data, code, outputs, traces, or system state to produce a conclusion, metric, summary, or diagnosis.
 - **Secure or harden** — Remove vulnerabilities, strengthen permissions, protect secrets, or enforce security requirements.
-

Artifact type taxonomy — The artifact type describes the main objects the agent must inspect, modify, create, or validate to resolve the issue. Assign all labels that reflect the central artifacts involved. Each label must appear in the metadata of each task.

- **Codebase** — A multi-file software project, library, application, repository, or service source tree.
- **Single script or program** — A standalone script, source file, CLI program, or small self-contained program.
- **Test suite or benchmark** — Unit tests, integration tests, fixtures, golden tests, benchmark harnesses, or evaluation suites.
- **Build system or package metadata** — Build files, dependency manifests, lockfiles, project metadata, packaging configuration, or CI build definitions.
- **Configuration file** — Settings files, environment files, service configs, YAML/TOML/JSON/INI configs, dotfiles, or runtime parameters.
- **Service or daemon** — A running or configured service, background process, web server, worker, daemon, or containerized service.
- **Container or virtual environment** — Docker images, Dockerfiles, Compose projects, virtual environments, sandboxes, or isolated runtime environments.
- **Database or structured store** — Relational databases, NoSQL stores, key-value stores, tables, schemas, migrations, or stored records.
- **Dataset or tabular file** — CSVs, TSVs, spreadsheets, parquet files, data frames, or tabular datasets.
- **Text or log file** — Plain-text files, logs, markdown, transcripts, raw text corpora, or line-oriented records.
- **Binary executable or library** — Compiled binaries, shared libraries, object files, bytecode files, or native executables.
- **Model or checkpoint** — ML model files, weights, checkpoints, tokenizers, embeddings, model configs, or inference artifacts.
- **Network endpoint or protocol artifact** — API endpoints, socket services, packet captures, HTTP exchanges, TLS certificates, or network-facing interfaces.
- **Repository history or version-control state** — Git history, branches, commits, tags, submodules, conflicts, diffs, patches, hooks, or repository metadata.
- **Security artifact** — Secrets, keys, certificates, tokens, policies, vulnerability samples, or security-relevant configuration.
- **Mathematical or scientific model** — Equation systems, simulation models, solver inputs, statistical models, or formal computational models.
- **Generated output artifact** — The final produced file such as a transformed dataset, patch, package, chart, JSON file, or exported result.